

# Conception d'une plateforme de réservation de terrain de padel intégrant le Model Context Protocol

TRAVAIL DE BACHELOR HES RÉALISÉ EN VUE DE  
L'OBTENTION DU BACHELOR PAR :

JONATHAN BOREL-JAQUET

CONSEILLERS AU TRAVAIL DE BACHELOR :  
LUDOVIC BERTIN

GENÈVE, LE 6 MAI 2026

Haute école de Gestion de Genève (HEG-GE)

Filière Informatique de gestion

# 1 Déclaration

Ce travail de Bachelor est réalisé dans le cadre de l'examen final de la Haute école de gestion de Genève, en vue de l'obtention du titre de Bachelor of Science HES-SO en Informatique de gestion.

L'étudiant a envoyé ce document par email à l'adresse remise par son conseiller au travail de Bachelor pour analyse par le logiciel de détection de plagiat URKUND, selon la procédure détaillée à l'URL suivante : <https://www.urkund.com>

L'étudiant accepte, le cas échéant, la clause de confidentialité. L'utilisation des conclusions et recommandations formulées dans le travail de Bachelor, sans préjuger de leur valeur, n'engage ni la responsabilité de l'auteur, ni celle du conseiller au travail de Bachelor, du juré et de la HEG.

"J'atteste avoir réalisé seul le présent travail, sans avoir utilisé des sources autres que celles citées dans la bibliographie."

Fait à Genève, le 6 mai 2026

Jonathan Borel-Jaquet

---

## 2 Remerciements

Remercier Hauri pour son aide précieuse au choix du sujet de TB. Remercier David Paulino pour son template LaTeX qui m'a permis de gagner un temps précieux lors de la rédaction de ce document. Remercier Ludovic Bertin pour son accompagnement tout au long de ce travail de Bachelor, ses conseils avisés et sa disponibilité. Remercier mes proches pour leur soutien moral et leur relecture attentive de ce document.

## 3 Résumé

## Liste des tableaux

|    |                                                                              |    |
|----|------------------------------------------------------------------------------|----|
| 1  | Priorisation détaillée des livrables selon la méthodologie MoS-CoW . . . . . | 18 |
| 2  | Planification Gantt du projet (Partie 1) . . . . .                           | 20 |
| 3  | Planification Gantt du projet (Partie 2) . . . . .                           | 21 |
| 4  | Dictionnaire de données - Table User . . . . .                               | 32 |
| 5  | Dictionnaire de données - Table Club . . . . .                               | 32 |
| 6  | Dictionnaire de données - Table Court . . . . .                              | 33 |
| 7  | Dictionnaire de données - Table Match . . . . .                              | 34 |
| 8  | Dictionnaire de données - Table Participant . . . . .                        | 35 |
| 9  | Vue d'ensemble des points de terminaison de l'API PadelContext               | 36 |
| 10 | Liste des outils exposés par le serveur MCP de PadelContext .                | 42 |

## Table des figures

|   |                                                                                               |    |
|---|-----------------------------------------------------------------------------------------------|----|
| 1 | Aperçu de l'application PadelConnect . . . . .                                                | 14 |
| 2 | Aperçu de l'application PlayPad . . . . .                                                     | 15 |
| 3 | Diagramme de cas d'utilisation de l'application PadelContext .                                | 27 |
| 4 | Diagramme de l'architecture logicielle de l'écosystème Padel-<br>Context . . . . .            | 30 |
| 5 | Schéma relationnel de la base de données de l'application Pa-<br>delConnect . . . . .         | 31 |
| 6 | Mécanisme de transport local via Stdio, adapté de (Dohnal,<br>2025) [15] . . . . .            | 39 |
| 7 | Mécanisme de transport réseau via Streamable HTTP, adapté<br>de (Dohnal, 2025) [15] . . . . . | 40 |

## Table des matières

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| <b>1</b> | <b>Déclaration</b>                                            | <b>1</b>  |
| <b>2</b> | <b>Remerciements</b>                                          | <b>2</b>  |
| <b>3</b> | <b>Résumé</b>                                                 | <b>3</b>  |
|          | Liste des tableaux                                            | 4         |
|          | Table des figures                                             | 5         |
| <b>4</b> | <b>Introduction</b>                                           | <b>8</b>  |
| 4.1      | Introduction à la problématique . . . . .                     | 8         |
| 4.2      | Intérêt de la problématique . . . . .                         | 9         |
| 4.3      | Questions auxquelles nous souhaitons répondre dans ce travail | 11        |
| 4.4      | Plan du document . . . . .                                    | 12        |
| <b>5</b> | <b>État de l’art</b>                                          | <b>12</b> |
| 5.1      | Solutions de réservation actuelles . . . . .                  | 12        |
| 5.1.1    | PadelConnect . . . . .                                        | 12        |
| 5.1.2    | PlayPad . . . . .                                             | 14        |
| 5.2      | Model Context Protocol . . . . .                              | 15        |
| <b>6</b> | <b>Méthodologie</b>                                           | <b>16</b> |
| 6.1      | MoSCoW . . . . .                                              | 16        |
| 6.2      | Gantt . . . . .                                               | 19        |
| <b>7</b> | <b>Analyse des besoins et choix technologiques</b>            | <b>21</b> |
| 7.1      | Backend . . . . .                                             | 21        |
| 7.1.1    | Choix de l’architecture de l’API . . . . .                    | 21        |
| 7.1.2    | Choix du système de base de données . . . . .                 | 22        |
| 7.1.3    | Choix du framework et des outils de développement . .         | 23        |
| 7.2      | Conteneurisation et gestion des environnements . . . . .      | 24        |
| 7.3      | Frontend . . . . .                                            | 26        |
| 7.4      | Identification des sources de données . . . . .               | 26        |
| 7.5      | Architecture du Model Context Protocol . . . . .              | 26        |

|           |                                                          |           |
|-----------|----------------------------------------------------------|-----------|
| <b>8</b>  | <b>Conception et architecture de l'application</b>       | <b>27</b> |
| 8.1       | Modélisation fonctionnelle . . . . .                     | 27        |
| 8.1.1     | Diagramme de cas d'utilisation . . . . .                 | 27        |
| 8.1.2     | Description des acteurs et des fonctionnalités . . . . . | 27        |
| 8.2       | Architecture logicielle et écosystème . . . . .          | 29        |
| 8.3       | Modélisation de la base de données . . . . .             | 31        |
| 8.3.1     | Schéma relationnel . . . . .                             | 31        |
| 8.3.2     | Dictionnaire de données . . . . .                        | 31        |
| 8.4       | Spécifications techniques de l'API REST . . . . .        | 35        |
| 8.4.1     | Documentation des points de terminaison . . . . .        | 35        |
| 8.4.2     | Documentation et stratégie de tests . . . . .            | 37        |
| 8.5       | Maquettes . . . . .                                      | 38        |
| 8.6       | Serveur MCP . . . . .                                    | 38        |
| 8.6.1     | Mécanisme de transport et communication . . . . .        | 38        |
| 8.6.2     | Définitions des outils MCP . . . . .                     | 41        |
| 8.7       | Cas d'usage . . . . .                                    | 44        |
| 8.8       | Comparaison des résultats . . . . .                      | 44        |
| <b>9</b>  | <b>Conclusion</b>                                        | <b>44</b> |
| 9.1       | Conclusion générale . . . . .                            | 44        |
| 9.2       | Améliorations possibles . . . . .                        | 44        |
|           | <b>Références</b>                                        | <b>44</b> |
| <b>10</b> | <b>Annexes</b>                                           | <b>47</b> |
| 10.1      | Diagramme gantt du projet PadelContext . . . . .         | 47        |
| 10.2      | Documentation Swagger de l'API PadelContext . . . . .    | 48        |

## 4 Introduction

### 4.1 Introduction à la problématique

Le padel est un sport de raquette dérivé du tennis qui se joue exclusivement en double<sup>1</sup>, sur un court de dimensions réduites et encadré de vitres ou de grillages. Au cours de ces dernières années, cette discipline a connu une croissance fulgurante à l'échelle mondiale, et plus particulièrement en Europe ainsi qu'en Amérique latine. Face à cet engouement massif, de nombreuses infrastructures ont été construites pour répondre à une demande toujours plus grandissante. Cependant, bien que l'accès à ce sport se démocratise, l'organisation et la réservation de ces terrains représentent encore aujourd'hui un véritable défi pour ses pratiquants.

Dans le cadre de ce travail de bachelor, nous allons nous intéresser plus précisément à la gestion et aux problématiques de réservation des terrains de padel dans le canton de Genève. Actuellement, l'offre genevoise se répartit sur plusieurs sites dont les plus connus sont les suivants :

- Centre sportif de Vessy : 1 terrain outdoor<sup>2</sup>
- Centre sportif universitaire (CSU) : 1 terrain outdoor
- Centre sportif de Bernex : 2 terrains outdoor
- Parc des Evaux : 3 terrains outdoor
- Padel Academy, Jonction : 2 terrains couverts
- Club international de tennis, Pregny-Chambésy : 2 terrains indoor<sup>3</sup>

Comme nous pouvons le constater, beaucoup de terrains de padel à Genève se situe en extérieur. Cette caractéristique amène un premier défi de taille : la dépendance à la météorologie. En effet, les conditions climatiques sont un facteur déterminant dans la pratique de ce sport. La pluie rend le jeu difficile, voire impossible, car elle modifie la physique de la balle et altère ses rebonds sur le sol et les vitres. Le vent dévie les trajectoires, tandis que l'éblouissement causé par le soleil complique fortement la lecture des trajectoires aériennes, notamment lors des lobs (nécessitant souvent l'anticipation avec des lunettes ou des casquettes). Bien que les terrains indoor permettent

---

1. Deux joueurs contre deux joueurs

2. Outdoor signifie que le terrain est situé à l'extérieur.

3. Indoor signifie que le terrain est situé à l'intérieur.

de s'affranchir de ces contraintes météorologiques, ils s'avèrent souvent plus coûteux, posant ainsi une barrière financière pour les joueurs disposant d'un budget limité. Outre les facteurs environnementaux, la nature même du padel génère des contraintes organisationnelles complexes. Ce sport nécessitant impérativement la présence de quatre joueurs, la recherche de partenaires devient une problématique centrale. Il ne s'agit pas seulement de faire coïncider les disponibilités horaires de quatre individus, mais également de s'assurer d'une homogénéité des niveaux pour garantir la qualité du jeu. Le système de réservation doit ainsi pouvoir répondre à des dynamiques de "matchmaking" variées : un joueur cherchant trois inconnus, un binôme cherchant deux adversaires, ou encore l'intégration d'un joueur inconnu pour compléter un groupe d'amis.

Enfin, l'aspect logistique ne s'arrête pas à la formation des équipes. De nombreux joueurs occasionnels ne possédant pas leur propre équipement, l'accès au matériel sur le lieu de pratique est primordial. La disponibilité de box de location (raquettes et balles) à proximité immédiate des terrains devient alors un critère de choix décisif lors de la réservation.

Au vu de la multitude de paramètres à prendre en compte : contraintes météorologiques, homogénéité des niveaux, flexibilité du matchmaking, budget et disponibilité du matériel, l'organisation d'une partie s'apparente aujourd'hui à un véritable casse-tête logistique. Le traitement simultané de toutes ces variables dépasse largement les capacités d'un système de réservation traditionnel. La problématique de ce travail de bachelor est donc de concevoir une application de réservation centralisée et intelligente utilisant une IA couplée au Mondel Context Protocol, spécifiquement adaptée aux contraintes du padel genevois pour réduire ces difficultés.

## 4.2 Intérêt de la problématique

Aujourd'hui, la réservation de terrains de padel à Genève est un véritable casse-tête organisationnel. En effet, la multitude de variables à prendre en compte rend la planification d'une partie particulièrement complexe.

Parmi ces contraintes, la météo joue un rôle déterminant. Tout pratiquant souhaitant réserver un terrain en extérieur doit systématiquement croiser les disponibilités avec des données météorologiques (via des applications comme

MeteoSwiss, par exemple) pour s'assurer de conditions de jeu favorables. À cela s'ajoute la difficulté de réunir quatre partenaires de niveau équivalent, ce qui s'avère fastidieux lorsque l'on manque de contacts ou que les agendas de chacun diffèrent. Par ailleurs, pour les joueurs non équipés, il est impératif de vérifier en amont si le centre sportif propose du matériel de location.

Actuellement, des applications comme PadelConnect ou PlayPad n'offrent pas de solution globale face à ces contraintes. Elles se concentrent presque exclusivement sur la location de l'infrastructure, délaissant la charge mentale liée à l'organisation et au suivi logistique de la partie.

L'intérêt de cette problématique est donc multiple. Premièrement, elle répond à une frustration réelle et croissante au sein de la communauté. Face à l'essor rapide du padel, la demande pour un outil centralisé et intelligent devient très intéressant. Une application dédiée permettrait non seulement de simplifier le processus, mais aussi d'améliorer considérablement l'expérience utilisateur grâce à des fonctionnalités spécifiques à ce sport.

Deuxièmement, cette démarche offre l'opportunité de concevoir une solution logicielle innovante, potentiellement transposable à d'autres sports collectifs. L'application pourrait révolutionner la manière dont les sportifs s'organisent, en rendant la planification plus fluide, rapide et accessible.

Surtout, la véritable plus-value de ce projet réside dans l'intégration d'une intelligence artificielle propulsée par le Model Context Protocol. Contrairement aux applications classiques qui se limitent à afficher une grille horaire statique et laissent l'utilisateur gérer la météo, le coût et ses partenaires, cette architecture transforme le système en un véritable assistant proactif. L'objectif de ce travail est de démontrer qu'une IA peut absorber cette lourde charge décisionnelle en se renseignant dynamiquement sur des données externes pour proposer, de manière autonome, une expérience de réservation optimisée et totalement inédite sur le marché actuel.

Finalement, en levant ces barrières organisationnelles, la résolution de cette problématique contribue directement à la démocratisation du padel. En facilitant son accès, un public plus large pourrait être encouragé à le pratiquer, soutenant ainsi son développement dans la région.

### 4.3 Questions auxquelles nous souhaitons répondre dans ce travail

Le but de ce travail est de répondre à trois grandes questions.

1. Comment l'intégration du Model Context Protocol permet-elle à une intelligence artificielle de s'affranchir des approximations du web pour s'appuyer sur des données météorologiques officielles et fiables ?
2. De quelle manière un système centralisé par IA peut-il automatiser le matchmaking en croisant dynamiquement le niveau des joueurs, leurs disponibilités et les contraintes logistiques ?
3. Dans quelle mesure le passage d'une interface de réservation statique à un assistant proactif réduit-il la charge mentale et le temps d'organisation pour les utilisateurs ?

Le premier objectif de ce travail est de démontrer la viabilité technique d'une réservation assistée par IA basée sur des données précises et officielles. Actuellement, la plupart des modèles d'intelligence artificielle utilisent des données météorologiques existantes sur le web, souvent approximatives ou obsolètes face à la volatilité des conditions locales. Dans un sport très sensible à la météo comme le padel outdoor, une telle approximation n'est pas acceptable. Il s'agira donc de comprendre comment l'architecture MCP<sup>4</sup> permet de forcer l'IA à interroger des sources de données officielles (via des API météorologiques certifiées) pour garantir que le choix du terrain (intérieur ou extérieur) soit optimal et sans risque.

Le deuxième objectif vise à résoudre le casse-tête organisationnel propre à ce sport. L'application devra prouver sa capacité à analyser une base de données d'utilisateurs pour automatiser la création de parties. Le but est de comprendre comment les algorithmes peuvent évaluer simultanément les niveaux relatifs des joueurs (pour garantir des matchs équilibrés), leurs disponibilités horaires croisées, et leurs besoins en matériel (Raquettes et/ou balles), afin de proposer le terrain et le créneau parfaits sans intervention manuelle fastidieuse.

---

4. Acronyme de Model Context Protocol

Enfin, le troisième objectif de ce travail est d'évaluer l'impact de cette solution sur l'expérience utilisateur (UX). En comparant le temps et les efforts nécessaires pour organiser une partie via les plateformes traditionnelles par rapport à notre solution intelligente propulsé par de l'intelligence artificielle.

## 4.4 Plan du document

# 5 État de l'art

## 5.1 Solutions de réservation actuelles

### 5.1.1 PadelConnect

PadelConnect est une application disponible sur Android, iOS et sur le web, qui permet de réserver des terrains de padel à Genève, plus précisément sur les quatre sites suivants :

- Centre sportif de Bernex
  - 2 terrains outdoor
  - Tous les jours de 07h30 à 22h30
  - 10 CHF par personne
- Parc des Evaux
  - 3 terrains outdoor
  - Tous les jours de 07h30 à 22h30
  - 10 CHF par personne
- Padel Academy, Jonction
  - 2 terrains couverts
  - Tous les jours de 09h00 à 21h30
  - ? CHF par personne
- Club international de tennis, Pregny-Chambésy
  - 2 terrains indoor
  - Tous les jours de 05h30 à 01h30
  - 17 CHF par personne

L'application ne stipule pas si le terrain désiré propose une box de location de matériel ou non, il est donc de la responsabilité de l'utilisateur de vérifier cette information avant de réserver un terrain.

Celle-ci propose une interface simple pour consulter les disponibilités des terrains et d'y effectuer des réservations pour des sessions de 1h30. Il est possible de réserver un terrain complet de manière privé pour y inviter des personnes petits à petits ou rejoindre une partie publique existante avec de parfait inconnue. Chaque réservation fournis sa propre page de discussion permettant aux joueurs de communiquer entre eux.

Concernant les données météorologiques, l'application ne propose pas de fonctionnalité pour vérifier les conditions météorologiques avant de réserver un terrain en extérieur. Il est donc de la responsabilité de l'utilisateur de vérifier les conditions météorologiques avant de réserver un terrain en extérieur. Toutefois, depuis le 27 octobre 2025, PadelConnect permet a ses utilisateurs d'annuler une partie et de se faire rembourser 180 minutes avant le début de cette dernière. À noter que la somme est remboursé sur le porte monnaie de l'utilisateur et non pas sur sa carte bancaire. Cette fonctionnalité permet aux utilisateurs de réserver un terrain en extérieur sans avoir à se soucier des conditions météorologiques, car ils peuvent annuler la réservation et se faire rembourser si les conditions météorologiques ne sont pas favorables.

L'applicaition permet également d'y renseigner son niveau de jeu allant de 1.0 à 10.0 par palier de 0.25 et son poste (Gauche, droite ou indifférent). Niveau totalement arbitraire et peu représentatif du niveau réel d'un joueur, mais qui permet tout de même de faire un semblant de matchmaking en sélectionnant des parties avec des joueurs de notre "estimation" de niveau.

Après une partie, l'application permet aussi d'y insérer les résultats de celle-ci. Cette dernière feature n'étant pas obligatoire, aucunement visible par les autres utilisateurs et non pris en compte dans le niveau du joueur, celle-ci est souvent ignorée par les utilisateurs.

Pour finir, l'application propose une fonctionnalité de recherche de joueurs grâce a différents filtros comme le niveau, la ville, l'age et le sexe.

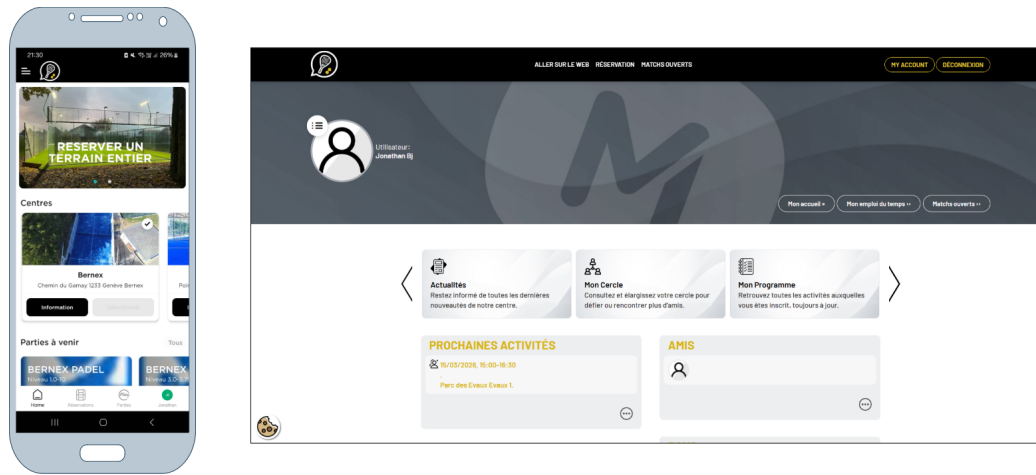


FIGURE 1 – Aperçu de l'application PadelConnect

### 5.1.2 PlayPad

PlayPad est une application fraîchement disponible sur Android et IOS. Actuellement, elle permet d'organiser des parties en Suisse Romande ainsi qu'en France voisine. Bien qu'elle n'ait pas encore été lancée officiellement, elle intègre des fonctionnalités intéressantes.

Contrairement à PadelConnect, la particularité de PlayPad est de se concentrer essentiellement sur le classement des joueurs. Pour l'instant, l'application ne permet pas de réserver directement un terrain, mais de choisir un terrain parmi ceux disponibles, cela est donc de la responsabilité du créateur de la partie de réserver le terrain choisie. À terme, l'application prévoit d'intégrer la réservation directe dans des clubs partenaires.

Comme mentionné précédemment, le cœur de l'application est son système de classement. En effet, les joueurs collectent des points au fil de leurs matchs pour grimper dans le classement général. Le niveau (Toujours sur une base de 1 à 10) n'est plus une simple auto-évaluation, mais dépend des résultats sur le terrain. « Chaque victoire et défaite influence le niveau du joueur » (PlayPad 2024) [1]. L'application introduit également un score de fiabilité (De 0 à 100%) qui fonctionne comme un score d'assiduité, plus le joueur est actif en mode compétitif, plus son pourcentage de fiabilité augmente.

On y retrouve un système de messagerie instantanée (via des demandes d'amis) ainsi qu'un chat dédié à chaque partie organisée.

Enfin, la saisie des résultats est publique, ce qui permet à l'ensemble de la communauté de suivre les performances de chacun.

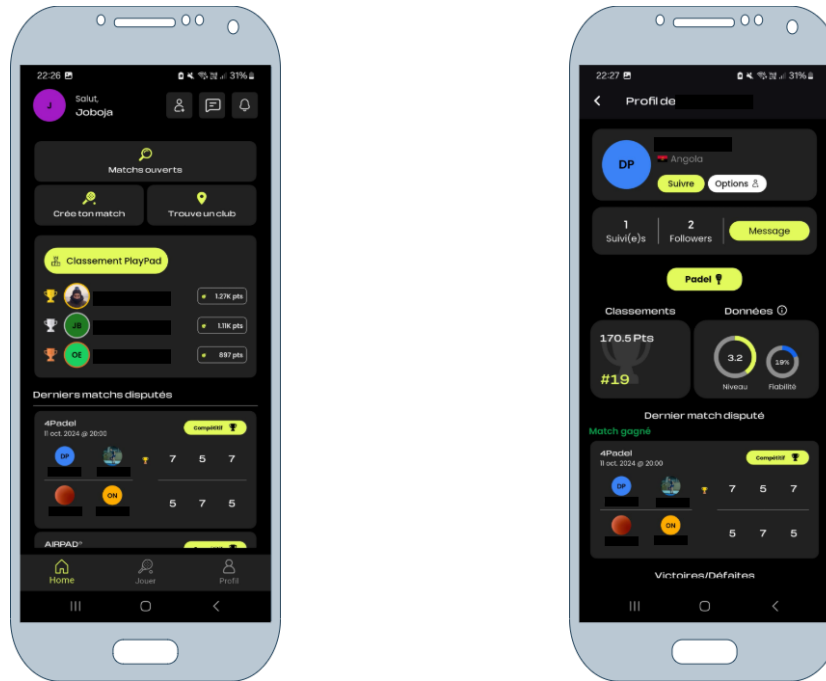


FIGURE 2 – Aperçu de l'application PlayPad

## 5.2 Model Context Protocol

Explication du concept de Model Context Protocol, son fonctionnement (Utilisation de données extenes), ses avantages et comment il peut être utilisé pour améliorer les systèmes de réservation.

## 6 Méthodologie

### 6.1 MoSCoW

Dans le cadre de ce travail de bachelor, je me suis appuyer sur la méthodologie MoSCoW afin de classer par ordre d'importance les différents livrables dont mon projet va introduire. Cette démarche m'a permis de définir un périmètre clair et précis avant le début de la phase de développement du projet. Ce périmètre m'a permis de gagner en efficacité et de me concentrer sur les éléments les plus importants pour la réussite de mon projet, tout en gardant une certaine flexibilité pour intégrer des fonctionnalités supplémentaires si le temps et les ressources le permettent.

La méthode MoSCoW est une technique de priorisation qui a gagné en popularité en raison de sa grande simplicité et de sa facilité d'adoption. Elle permet aux parties prenantes de gérer les attentes et les compromis techniques, tout en se concentrant sur la livraison des éléments les plus importants en classant les fonctionnalités dans les quatre catégories suivantes :

- **Must-have (Indispensable)** : Les exigences qui comportent des spécifications vitales à la réussite du projet et qui doivent être satisfaites dans les délais impartis. Si ces éléments ne sont pas mis en œuvre, le projet échouera ou aura un impact négatif majeur sur les utilisateurs (trad. de Vijayakumar, Prasad K, Holla M 2024, p. 2) [14].
- **Should-have (Souhaitable)** : Prérequis importants qui sont attrayants mais pas indispensables à la réussite immédiate du projet. Ils ajoutent de la valeur globale au produit et devraient être inclus une fois que toutes les fonctionnalités essentielles ont été satisfaites (trad. de Vijayakumar, Prasad K, Holla M 2024, p. 3) [14].
- **Could-have (Possible)** : Critères supplémentaires qui améliorent le produit mais ne sont pas nécessaires à ses fonctionnalités principales. Ces éléments ne sont pas prioritaires par rapport aux éléments indispensables ou souhaitables, mais ils peuvent être réalisés si le temps et les ressources le permettent (trad. de Vijayakumar, Prasad K, Holla M 2024, p. 3) [14].
- **Won't-have (Non souhaitable pour le moment)** : Besoins qui sont spécifiquement exclus du périmètre actuel du projet. Ils sont soit

considérés comme peu prioritaires pour le cycle de développement en cours, soit reportés à des versions ultérieures (trad. de Vijayakumar, Prasad K, Holla M 2024, p. 3) [14].

Dans l'objectif de la réalisation de la méthode MoSCoW, je me suis appuyé sur les recommandations du *DSDM Agile Project Framework Handbook* de l'Agile Business Consortium. Parmi ces recommandations, la plus déterminante concerne la répartition stratégique de l'effort de développement.

Afin de garantir la viabilité du projet et de se prémunir contre les imprévus, le référentiel préconise que les exigences classées comme indispensables (Must-have) ne doivent pas excéder 60 % de l'effort global alloué au projet. Les 40 % restants agissent comme une marge de contingence et sont alloués aux fonctionnalités souhaitables (Should-have) et possibles (Could-have), généralement réparties à parts égales soit 20 % chacune (Agile Business Consortium 2014) [8].

L'application stricte de cette règle permet d'assurer qu'en cas de complexité sous-estimée ou de contraintes de temps inattendues sur les éléments critiques, le travail de bachelor pourra toujours être livré dans les délais fixés, quitte à sacrifier temporairement des fonctionnalités de moindre priorité. C'est sur la base de ce cadre temporel et de ces pourcentages d'effort que le tableau suivant a été construit.

| ID | Fonctionnalité / Tâche                                                                                                       | Catégorie   | Effort |
|----|------------------------------------------------------------------------------------------------------------------------------|-------------|--------|
| 01 | Modélisation de la base de données et création de l'API de base                                                              | Must-have   | 15%    |
| 02 | Mise en place du serveur MCP et intégration du modèle IA                                                                     | Must-have   | 10%    |
| 03 | Outil MCP (BDD) : Recherche filtrée des infrastructures (localisation basique par texte, prix, indoor/outdoor, box matériel) | Must-have   | 10%    |
| 04 | Outil MCP (BDD) : Matchmaking (recherche de parties ouvertes selon le niveau et les places disponibles)                      | Must-have   | 10%    |
| 05 | Outil MCP (API) : Interrogation des prévisions météorologiques locales pour la validation des terrains extérieurs            | Must-have   | 10%    |
| 06 | Outil MCP (BDD) : Exécution de la réservation (verrouillage du créneau et intégration des joueurs)                           | Must-have   | 5%     |
| 07 | Création d'une application web minimaliste pour la gestion des réservations                                                  | Should-have | 10%    |
| 08 | Étude comparative entre les plateformes traditionnelles et la solution MCP                                                   | Should-have | 10%    |
| 09 | Application mobile native (iOS / Android)                                                                                    | Could-have  | 10%    |
| 10 | Outil MCP (BDD) : Calculs avancés du niveau réel des joueurs par rapport à leur historique de parties                        | Could-have  | 5%     |
| 11 | Outil MCP (Géolocalisation) : Filtrage avancé des terrains par coordonnées GPS                                               | Could-have  | 5%     |
| 12 | Interface de messagerie instantanée entre les joueurs                                                                        | Won't-have  | -      |
| 13 | Paieement intégré lors de la réservation d'un terrain                                                                        | Won't-have  | -      |

TABLE 1 – Priorisation détaillée des livrables selon la méthodologie MoSCoW

## 6.2 Gantt

Dans la continuité de la méthodologie MoSCoW, la réalisation d'une planification Gantt du projet m'a permis d'organiser et visualiser l'ensemble des tâches à réaliser, leur ordre chronologique et leur durée estimée. Elle a également permis d'identifier les dépendances entre les différentes tâches et à anticiper les éventuels retards ou obstacles qui auraient pu survenir au cours du projet.

Les jalons, identifiés par un losange noir (◆), représentent les points clés du projet, tels que la mise en place d'une base de données et d'une API opérationnelles, ou encore la capacité du serveur MCP à interroger les données internes de l'application et l'API MeteoSwiss. Ces jalons sont essentiels pour mesurer l'avancement du projet et s'assurer que les objectifs intermédiaires sont atteints avant de passer à la phase suivante.

Le diagramme de Gantt est disponible dans l'annexe 10.1.

| Nom de la tâche                                                                        | Date de début   | Date de fin     |
|----------------------------------------------------------------------------------------|-----------------|-----------------|
| Introduction de la documentation                                                       | 02.03.26        | 06.03.26        |
| Méthodologie et planification                                                          | 09.03.26        | 13.03.26        |
| <b>Backend</b>                                                                         | <b>16.03.26</b> | <b>10.04.26</b> |
| Modélisation et documentation de la base de données                                    | 16.03.26        | 20.03.26        |
| Étude, choix et documentation des technologies backend                                 | 23.03.26        | 27.03.26        |
| Création et mise en place du serveur backend                                           | 30.03.26        | 03.04.26        |
| Documentation de l'architecture backend                                                | 06.04.26        | 10.04.26        |
| <b>♦ Base de données et API opérationnelles</b>                                        | <b>13.04.26</b> | <b>13.04.26</b> |
| <b>MCP - Base de données</b>                                                           | <b>13.04.26</b> | <b>24.04.26</b> |
| Documentation et création des tools MCP avec les données internes                      | 13.04.26        | 17.04.26        |
| Création du MVP pour tester les tools                                                  | 20.04.26        | 24.04.26        |
| <b>♦ Le serveur MCP est capable d'interroger les données internes de l'application</b> | <b>27.04.26</b> | <b>27.04.26</b> |
| <b>MCP - API MeteoSwiss</b>                                                            | <b>27.04.26</b> | <b>08.05.26</b> |
| Documentation et création des tools MCP avec l'API MeteoSwiss                          | 27.04.26        | 01.05.26        |
| Création du MVP pour tester les tools                                                  | 04.05.26        | 08.05.26        |
| <b>♦ Le serveur MCP est capable d'interroger l'API MeteoSwiss</b>                      | <b>11.05.26</b> | <b>11.05.26</b> |

TABLE 2 – Planification Gantt du projet (Partie 1)

| Nom de la tâche                                                   | Date de début   | Date de fin     |
|-------------------------------------------------------------------|-----------------|-----------------|
| <b>Frontend</b>                                                   | <b>11.05.26</b> | <b>22.05.26</b> |
| Documentation de l'architecture générale                          | 11.05.26        | 15.05.26        |
| Création de l'application WEB minimaliste                         | 18.05.26        | 22.05.26        |
| <b>♦ Prototype complet et testable</b>                            | <b>25.05.26</b> | <b>25.05.26</b> |
| <b>Étude comparative</b>                                          | <b>25.05.26</b> | <b>29.05.26</b> |
| Rédaction de l'analyse comparative (PadelContext vs PadelConnect) | 25.05.26        | 29.05.26        |
| <b>♦ Étude complétée et documentée</b>                            | <b>01.06.26</b> | <b>01.06.26</b> |
| <b>Fonctionnalités optionnelles et conclusion</b>                 | <b>01.06.26</b> | <b>19.06.26</b> |
| Développement de l'application mobile native PadelContext         | 01.06.26        | 05.06.26        |
| Documentation de la conclusion et des améliorations possibles     | 08.06.26        | 12.06.26        |
| Finalisation de la documentation                                  | 15.06.26        | 19.06.26        |
| <b>♦ Rendu final du Travail de Bachelor</b>                       | <b>22.06.26</b> | <b>22.06.26</b> |

TABLE 3 – Planification Gantt du projet (Partie 2)

## 7 Analyse des besoins et choix technologiques

### 7.1 Backend

#### 7.1.1 Choix de l'architecture de l'API

Dans l'objectif d'assurer l'échange de données entre les interfaces utilisateur et la base de données, tout en fournissant un point d'accès standardisé au serveur MCP, le développement d'une API s'est avéré indispensable.

Pour ce faire, il a d'abord fallu déterminer l'architecture d'API la plus adaptée aux contraintes du projet. Parmi les standards actuels les plus connus se trouvent les architectures SOAP, REST et GraphQL. Dans le cadre de

ce projet, j'ai choisi d'utiliser l'architecture REST. En effet, contrairement à l'architecture SOAP qui repose sur des messages XML et une structure de communication plus rigide, l'architecture REST prône la simplicité et une communication sans état.

De plus, REST s'intègre parfaitement aux standards WEB en privilégiant fortement l'utilisation de format de données légers tels que JSON, ce qui en fait aujourd'hui le standard pour la conception d'API web. Cette légèreté est idéale pour garantir des temps de réponse rapides entre ma base de données, mes application et mon serveur MCP.

Enfin, bien que GraphQL émerge actuellement comme un leader pour la conception d'API dans les applications hautement dynamiques nécessitant des requêtes de données très complexes, son implémentation dans le cadre de ce travail de bachelor aurait ajouté une complexité superflue. Les opérations liées à une application de réservation de terrain de padel suivent une logique transactionnelle standard pour laquelle la simplicité éprouvée de l'architecture REST est amplement suffisante et parfaitement adaptée (Addanki 2025) [7].

### 7.1.2 Choix du système de base de données

Afin de stocker et gérer efficacement les données nécessaires à mon application et dans le but de permettre à mon API REST d'écrire et fournir des données, il a fallu choisir quel type de système de base de données utiliser entre une base de données relationnelle SQL et une base de données non relationnelle NoSQL.

Le choix le plus judicieux a été une base de données relationnelle SQL. En effet, les bases de données relationnelles sont particulièrement adaptées pour gérer des données structurées avec des relations complexes entre les différentes entités, ce qui est le cas dans une application de réservation de terrains de padel. Les données telles que les utilisateurs, les réservations, les matchs, les terrains, etc., sont toutes interconnectées et nécessitent une gestion rigoureuse des relations entre elles.

De plus, le respect des propriétés ACID (Atomicité, Cohérence, Isolation, Durabilité) inhérent aux bases de données relationnelles permet de garantir

une fiabilité et une intégrité absolues des données lors des transactions. Dans le contexte d'une plateforme de réservation, cela s'avère indispensable pour gérer les accès concurrents, comme s'assurer que deux groupes de joueurs ne puissent pas réserver le même terrain au même instant, ou garantir qu'une opération d'enregistrement s'annule complètement si une erreur survient en cours de processus.

« Dès lors que les propriétés ACID sont indispensables pour une base de données, il ne fait aucun doute que le seul choix possible est la base de données relationnelle. » (trad. de Sahatqija et al. 2018, p. 5) [12].

Une fois le choix le type de système de base de données effectué, il a fallu choisir quel SGBDR (Système de Gestion de Base de Données Relationnel) utiliser. Parmi les deux options open-source les plus populaires se trouvent MySQL et PostgreSQL.

Suite à l'analyse d'une étude récente de 2024 comparant les performances de ces deux moteurs de base de données, mon choix s'est définitivement porté sur PostgreSQL. En effet, cette recherche a mis en évidence la performance significative de PostgreSQL par rapport à MySQL à travers plusieurs tests simulant des conditions de production. Les résultats ont notamment démontré que le temps d'exécution pour la récupération d'une très grande quantité d'enregistrements était environ 13 fois plus rapide avec PostgreSQL qu'avec MySQL. Par ailleurs, pour les requêtes de sélection intégrant une clause de recherche spécifique, PostgreSQL s'est également révélé être environ 9 fois plus efficace MySQL (Sanket Vilas et Ouda 2024, p. 1) [13].

### 7.1.3 Choix du framework et des outils de développement

Pour clôturer les choix technologiques liés à la conception du backend, il a fallu déterminer le framework à utiliser pour développer l'API REST. Mon choix s'est porté sur l'environnement Node.js associé au framework Express.js. Cette décision s'articule autour de trois critères majeurs.

Le premier critère repose sur le fait d'avoir déjà eu l'opportunité d'utiliser Express.js lors d'un projet de développement sur mandat au cours de mes études à la HEG, possédant une bonne maîtrise de ce framework et du langage JavaScript, cela me permet de développer l'API plus efficacement.

Le second critère concerne la popularité du langage JavaScript. En effet, selon l'enquête de 2025 de Stack Overflow, JavaScript demeure le langage de programmation le plus utilisé sur la plateforme (Stack Overflow 2025) [3], Stack Overflow étant un forum largement répandu et connu, la consultation de celui-ci est avantageuse lors de la réalisation d'un projet académique de cet envergure.

Afin de gagner en efficacité et de réduire le temps de développement de l'API REST, j'ai également choisi d'utiliser Prisma pour la gestion de ma base de données. Cet ORM (Object-Relational Mapping) simplifie considérablement les interactions avec la base de données en offrant une interface de programmation intuitive. Il permet d'automatiser des tâches chronophages et critiques telles que la validation des données, la gestion des migrations structurelles, et la génération de requêtes SQL optimisées et sécurisées.

Au-delà du confort de développement, ce choix s'appuie sur des critères de performance concrets. Prisma offre les meilleures performances lorsqu'il est soumis à des charges parallèles par rapport à des alternatives comme Sequelize ou TypeORM (Zhadko-Bazilevych 2025) [16]. Pour une plateforme de réservation comme PadelContext, où plusieurs utilisateurs peuvent solliciter l'API simultanément pour rechercher ou réserver un créneau via le serveur MCP, cette capacité à maintenir de hautes performances sous une charge parallèle est un atout technique décisif.

Enfin, pour assurer la fiabilité de mon API REST, j'ai opté pour le framework de test Jest. Jest est un framework qui ne nécessite aucune configuration au préalable. De plus, il fournit un système de mocks intégré ainsi qu'une couverture de code complète (Donvir 2024) [9]. De manière similaire à Prisma, le choix de Jest est principalement justifié par sa rapidité d'intégration et son efficacité, dans l'objectif d'optimiser le temps alloué au développement de l'API REST.

## 7.2 Conteneurisation et gestion des environnements

Dans le cadre de ce projet, la conteneurisation de l'intégralité de l'écosystème applicatif s'est imposée comme une nécessité technique, motivée par des enjeux de reproductibilité et de facilité de déploiement. Pour répondre à ce besoin, j'ai choisis d'utiliser la bibliothèque et plateforme logicielle Docker.

Le fonctionnement de Docker repose sur l'utilisation d'images. Les images Docker sont des modèles permettant de créer un conteneur Docker, qui intègrent toutes les dépendances nécessaires au fonctionnement des applications. Les images Docker sont des conteneurs Docker qui exécutent des applications dans des environnements isolés, où le résultat doit être identique quelle que soit l'infrastructure sous-jacente (trad. de Naga Murali Krishna 2025, p.725) [11].

Cette isolation et cette reproductibilité générale garanties par Docker ont été les principaux arguments de son adoption pour PadelContext. La capacité de pouvoir développer, tester et déployer l'application sur différents appareils sans se soucier du système d'exploitation hôte ou de ses configurations spécifiques s'est avérée extrêmement pratique.

De plus, afin d'optimiser les ressources, chaque conteneur de l'écosystème repose sur une image de base minimaliste de type Alpine Linux. Ces images allégées ne mettent à disposition que le strict nécessaire au fonctionnement de l'application, ce qui réduit considérablement la surface d'attaque potentielle et accélère le temps de déploiement des conteneurs.

Afin de prévenir l'obsolescence prématurée du projet et de minimiser les problèmes de compatibilité durant la phase de développement, un choix strict de sélection des versions a été appliquée pour chaque image Docker. Le choix s'est systématiquement porté sur des versions actives, stables et officiellement maintenues, garantissant ainsi l'accès à une documentation à jour en cas de difficulté de codage.

Pour les images de l'API REST, du serveur MCP et du client MCP fonctionnant tous avec un environnement d'exécution JavaScript avec Node.js, l'image Docker `node:24.14.0-alpine3.23` a été sélectionnée. Ce choix est justifié par les recommandations officielles de Node.js, qui spécifient que la version 24 est la version *LTS (Long-Term Support)* durant la période de réalisation de ce travail de Bachelor. Ce statut LTS garantit la correction des failles de sécurité et des bogues critiques pendant une durée de 30 mois (Node.js, 2025) [5].

Enfin, pour la base de données PostgreSQL, l'image `postgres:18.3-alpine`

a été retenue. Ce choix suit la même logique de pérennité en s'appuyant sur les recommandations officielles de PostgreSQL, qui identifient la version 18 comme la version majeure la plus récente et supportée à ce jour (PostgreSQL, 2025) [2].

### **7.3 Frontend**

### **7.4 Identification des sources de données**

API MeteoSwiss et base de données de l'application

### **7.5 Architecture du Mondel Context Protocol**

Comment l'IA va interagir avec les différentes sources de données ?

## 8 Conception et architecture de l'application

### 8.1 Modélisation fonctionnelle

#### 8.1.1 Diagramme de cas d'utilisation

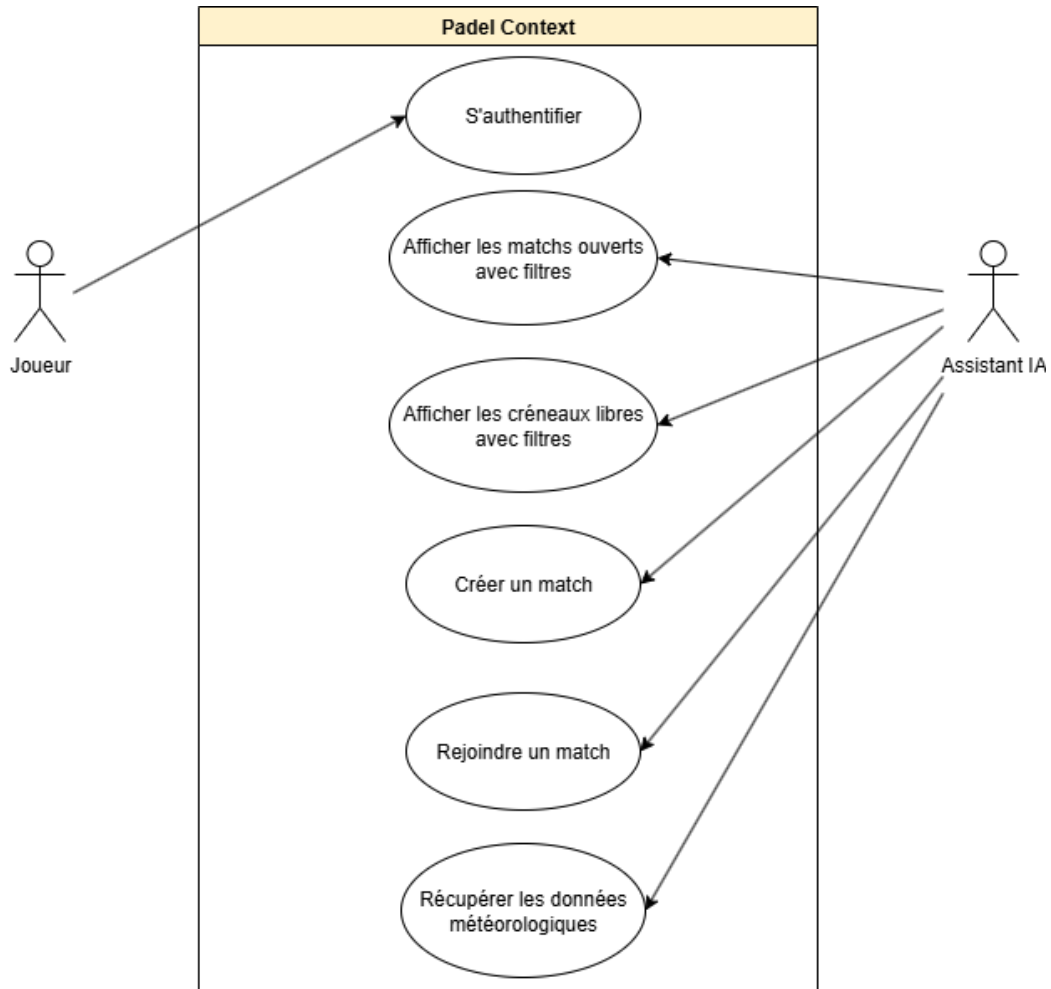


FIGURE 3 – Diagramme de cas d'utilisation de l'application PadelContext

#### 8.1.2 Description des acteurs et des fonctionnalités

Afin de bien délimiter le périmètre d'action de l'application PadelContext, il est essentiel de définir les différents acteurs interagissant avec le système,

ainsi que les scénarios d'utilisation principaux qui en découlent. Contrairement à une application classique, l'intégration du Model Context Protocol modifie l'interaction entre les acteurs. L'utilisateur ne se renseigne pas et ne navigue pas uniquement via des clics dans des menus, mais dialogue avec un assistant IA qui orchestre les requêtes de manière proactive.

Les acteurs principaux de l'application sont les suivants :

- **Le Joueur** : Il s'agit de l'utilisateur final de l'application. Son objectif est de trouver des partenaires, de consulter la disponibilité des terrains et de planifier des parties de padel tout en s'affranchissant de la charge mentale logistique (météo, équipement sur place, niveau des autres joueurs, etc.).
- **L'Assistant IA** : Utilisant un LLM et connecté au serveur MCP, il agit comme un assistant proactif. Il comprend les requêtes en langage naturel du joueur et utilise les outils mis à sa disposition pour interroger l'API REST de l'application ainsi que l'API externe MeteoSwiss.

Les scénarios d'utilisation principaux sont les suivants :

- **Connexion** : Le joueur s'authentifie sur l'application. Cette étape permet de personnaliser ses futures requêtes auprès de l'Assistant IA et de l'autoriser à exécuter des actions nécessitant une identification (comme rejoindre une partie ou réserver un terrain).
- **Afficher les parties existantes** : Le joueur exprime à l'Assistant IA son désir de participer à un match, par exemple : « Je cherche à rejoindre une partie pour demain à 18h00 avec des joueurs d'un niveau moyen de 5.0 ». L'Assistant IA fait appel au serveur MCP pour interroger l'outil approprié. Il analyse les matchs ouverts et filtre les résultats pour ne proposer que ceux correspondant au niveau du joueur et à la tranche horaire souhaitée.
- **Rejoindre une partie existante** : Si le joueur valide la proposition de l'Assistant IA, ce dernier vérifie l'état de connexion du joueur, puis exécute l'action nécessaire pour l'inscrire à la partie sélectionnée.
- **Afficher les terrains disponibles** : Le joueur formule le souhait de réserver un terrain pour une date et une heure précises, par exemple : « Je souhaite réserver un terrain extérieur pour samedi prochain à 14h00 ». L'Assistant IA interroge le serveur MCP pour lister les ter-

rains disponibles et filtre les résultats selon la tranche horaire et le type de terrain demandés. De plus, avant de finaliser sa proposition, l'Assistant IA consulte l'API MeteoSwiss pour s'assurer que les conditions climatiques seront favorables.

- **Réserver un terrain** : Une fois la proposition validée par le joueur, l'Assistant IA vérifie que l'utilisateur est connecté et procède à la réservation effective du terrain.

## 8.2 Architecture logicielle et écosystème

L'architecture de PadelContext est conçue comme un système distribué où chaque fonctionnalité est isolée dans son propre environnement d'exécution. Grâce à son propre réseau virtuel privé, les différents conteneurs Docker qui composent l'écosystème de l'application peuvent communiquer entre eux de manière sécurisée et efficace.

Les ports utilisés pour les communications entre les conteneurs sont fixés en dur dans le code de chaque service, tandis que les ports exposés sur la machine hôte sont configurables via des variables d'environnement. Cette approche permet de maintenir une structure interne rigide afin de faciliter l'interaction entre les services pendant le développement de ce projet.

Afin d'illustrer cette architecture, le diagramme de la figure 4 synthétise les différents services et leurs interactions.

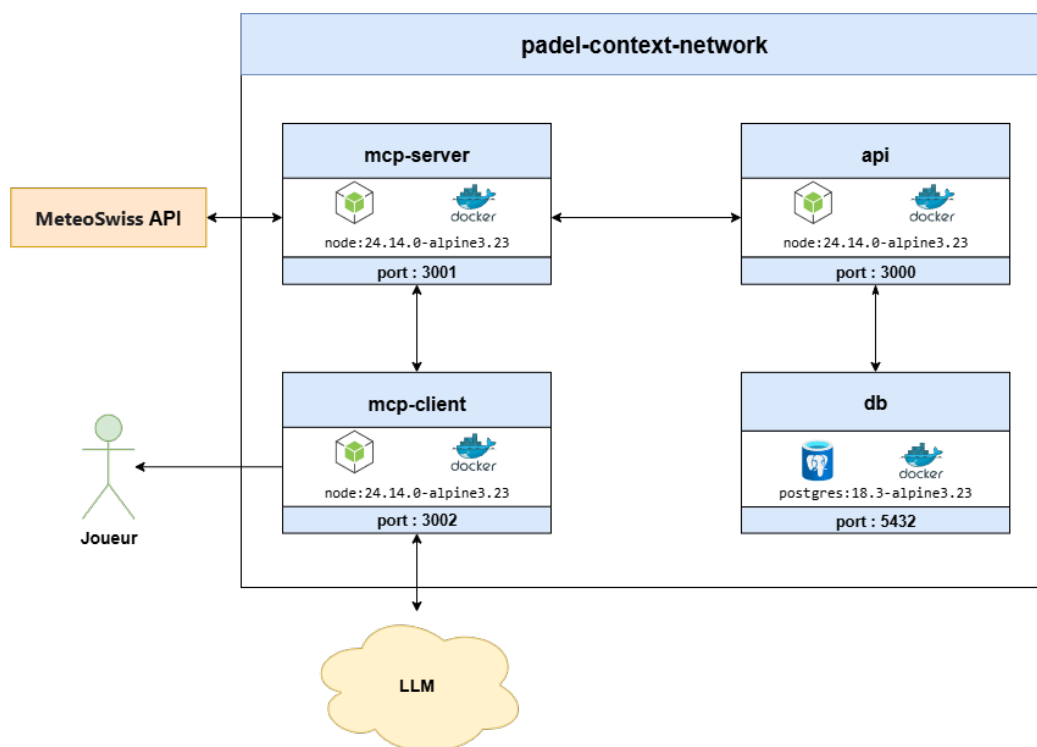


FIGURE 4 – Diagramme de l'architecture logicielle de l'écosystème Padel-Context

## 8.3 Modélisation de la base de données

### 8.3.1 Schéma relationnel

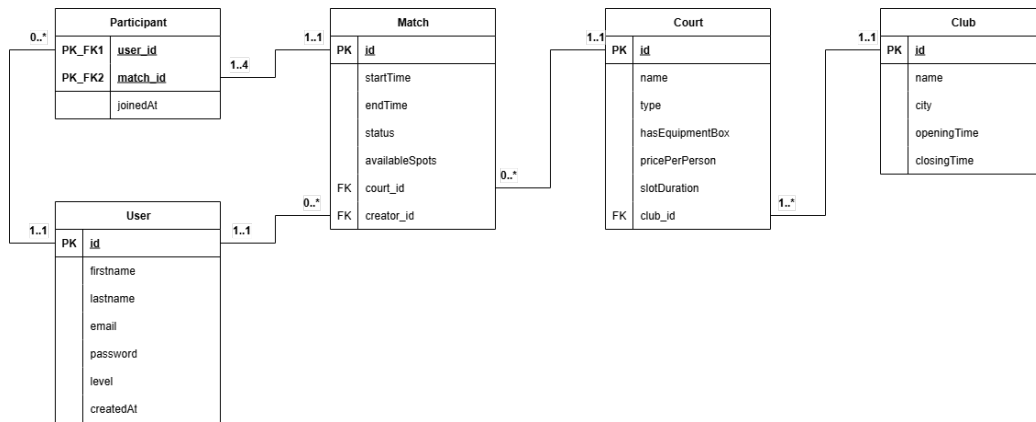


FIGURE 5 – Schéma relationnel de la base de données de l’application PadelConnect

### 8.3.2 Dictionnaire de données

Le dictionnaire de données ci-dessous détaille la structure des tables de la base de données relationnelle (PostgreSQL) générée par l’ORM Prisma. Il est important de noter la présence de deux énumérations (*Enums*) utilisées pour restreindre les valeurs de certains champs :

- **CourtType** : Définit le type de terrain (INDOOR, OUTDOOR, COVERED).
- **matchStatus** : Définit l’état actuel d’une partie (OPEN, COMPLETED, CANCELED).

| User      |          |                    |                                                |
|-----------|----------|--------------------|------------------------------------------------|
| Attribut  | Type     | Contraintes / Clés | Description                                    |
| id        | Int      | PK, Auto-incrément | Identifiant unique du joueur.                  |
| firstname | String   | Requis             | Prénom du joueur.                              |
| lastname  | String   | Requis             | Nom de famille du joueur.                      |
| email     | String   | Requis, Unique     | Adresse e-mail.                                |
| password  | String   | Requis             | Mot de passe haché.                            |
| level     | Int      | Défaut : 1         | Niveau du joueur.                              |
| createdAt | DateTime | Défaut : now()     | Date et heure d'inscription sur la plateforme. |

TABLE 4 – Dictionnaire de données - Table User

| Club        |        |                    |                                            |
|-------------|--------|--------------------|--------------------------------------------|
| Attribut    | Type   | Contraintes / Clés | Description                                |
| id          | Int    | PK, Auto-incrément | Identifiant unique du club.                |
| name        | String | Requis             | Nom du club.                               |
| city        | String | Requis             | Ville où se situe le club.                 |
| openingTime | String | Requis             | Heure d'ouverture habituelle quotidienne.  |
| closingTime | String | Requis             | Heure de fermeture habituelle quotidienne. |

TABLE 5 – Dictionnaire de données - Table Club

| <b>Court</b>    |             |                           |                                                      |
|-----------------|-------------|---------------------------|------------------------------------------------------|
| <b>Attribut</b> | <b>Type</b> | <b>Contraintes / Clés</b> | <b>Description</b>                                   |
| id              | Int         | PK, Auto-incrément        | Identifiant unique du terrain.                       |
| name            | String      | Requis                    | Nom d'identification du terrain.                     |
| type            | CourtType   | Défaut : OUTDOOR          | Type du terrain.                                     |
| hasEquipmentBox | Boolean     | Requis                    | Indique si le terrain met du matériel à disposition. |
| pricePerPerson  | Float       | Requis                    | Prix de la réservation par joueur.                   |
| slotDuration    | Int         | Requis                    | Durée standard d'un créneau de jeu en minutes.       |
| club_id         | Int         | FK                        | Référence au club propriétaire du terrain.           |

TABLE 6 – Dictionnaire de données - Table Court

| Match          |             |                    |                                                        |
|----------------|-------------|--------------------|--------------------------------------------------------|
| Attribut       | Type        | Contraintes / Clés | Description                                            |
| id             | Int         | PK, Auto-incrément | Identifiant unique de la partie.                       |
| startTime      | DateTime    | Requis             | Date et heure précises du début du match.              |
| endTime        | DateTime    | Requis             | Date et heure précises de la fin du match.             |
| status         | matchStatus | Défaut : OPEN      | État actuel du match.                                  |
| availableSpots | Int         | Requis             | Nombre de places encore disponibles pour ce match.     |
| court_id       | Int         | FK                 | Référence au terrain sur lequel se joue le match.      |
| creator_id     | Int         | FK                 | Référence au joueur ayant initié la création du match. |

TABLE 7 – Dictionnaire de données - Table Match

| Participant |          |                    |                                                       |
|-------------|----------|--------------------|-------------------------------------------------------|
| Attribut    | Type     | Contraintes / Clés | Description                                           |
| user_id     | Int      | PK, FK             | Identifiant du joueur rejoignant le match.            |
| match_id    | Int      | PK, FK             | Identifiant du match rejoint.                         |
| joinedAt    | DateTime | Défaut : now()     | Date et heure de l'inscription du joueur à la partie. |

TABLE 8 – Dictionnaire de données - Table Participant

## 8.4 Spécifications techniques de l'API REST

### 8.4.1 Documentation des points de terminaison

Afin de garantir une maintenance aisée et une compréhension rapide de l'API, l'ensemble des points de terminaison a été rigoureusement documenté en utilisant l'outil Swagger.

L'avantage de l'outil Swagger est qu'il permet de rédiger une documentation dynamique et accessible directement depuis le serveur de l'API. En effet, la documentation est générée automatiquement à partir de commentaires structurés directement intégrés au-dessus du code de chaque route de l'application. Cette méthode professionnelle garantit que le code et sa documentation restent constamment synchronisés.

L'API PadelContext expose cinq points de terminaison permettant de gérer l'authentification, la consultation des matchs ouverts et des créneaux horaires disponibles et leur participation ou création. Le tableau 9 présente une vue d'ensemble de ces routes.

| Méthode     | Endpoint                                 | Description                                                                                                                                |
|-------------|------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>POST</b> | <code>/api/auth/login</code>             | Authentifie un utilisateur via ses identifiants et retourne un JSON Web Token (JWT) valide pour authentifier ses futures requêtes.         |
| <b>GET</b>  | <code>/api/matches</code>                | Récupère la liste des matchs ouverts (en attente de joueurs) en appliquant divers filtres optionnels (ville, prix, durée, etc.).           |
| <b>GET</b>  | <code>/api/available-slots</code>        | Retourne les créneaux horaires disponibles pour les 7 prochains jours, en appliquant divers filtres optionnels (ville, prix, durée, etc.). |
| <b>POST</b> | <code>/api/matches/{matchId}/join</code> | Permet à un utilisateur de s'inscrire à une partie existante possédant encore des places disponibles grâce à son JWT.                      |
| <b>POST</b> | <code>/api/matches/from-slot</code>      | Permet à un utilisateur de convertir un créneau libre en un nouveau match ouvert et y inscrit automatiquement le créateur grâce à son JWT. |

TABLE 9 – Vue d'ensemble des points de terminaison de l'API PadelContext

La spécification technique complète et exhaustive de ces points de terminaison, incluant les paramètres de requête, le format des corps JSON attendus, ainsi que la gestion des différents codes de statut HTTP, est visualisable via l'interface graphique de Swagger.

Des captures d'écran détaillant l'utilisation et les réponses de chaque point de terminaison via cette interface sont disponibles dans l'annexe 10.2.

#### 8.4.2 Documentation et stratégie de tests

Afin de garantir la fiabilité, la robustesse et la non-régression de l'API REST tout au long de son développement, une stratégie de test rigoureuse a été mise en place. En accord avec les choix technologiques définis précédemment, le framework Jest a été utilisé pour orchestrer ces vérifications. La stratégie adoptée repose sur deux niveaux de validation complémentaires : les tests unitaires et les tests d'intégration.

Les tests unitaires ont pour objectif de valider le comportement d'un composant de code de manière totalement isolée du reste du système. Dans le cadre de l'API PadelContext, ces tests se concentrent exclusivement sur la logique métier des contrôleurs. Pour isoler efficacement chaque contrôleur, toutes les dépendances externes ont été simulées. Par exemple, les interactions avec la base de données via Prisma, le hachage des mots de passe avec la bibliothèque `bcryptjs` ou encore la génération de tokens avec la bibliothèque `jsonwebtoken` ont été remplacés par de fausses implémentations contrôlées par Jest. L'intérêt principal de cette approche est sa rapidité d'exécution et sa grande précision. En simulant des erreurs liées à la base de données, comme l'échec d'une transaction ou une perte de connexion, il a été possible de s'assurer que l'API gère correctement les exceptions et renvoie les bons codes HTTP sans avoir besoin d'altérer un environnement réel.

Contrairement aux tests unitaires, les tests d'intégration visent à vérifier que l'ensemble des composants du système communiquent correctement entre eux. Ils valident le cheminement complet d'une requête HTTP : du routeur au middleware permettant la vérification du JWT, en passant par le contrôleur, jusqu'à l'écriture ou la lecture dans une véritable base de données PostgreSQL. Ces tests ont été réalisés à l'aide de la bibliothèque Supertest, qui permet de simuler des requêtes HTTP réelles vers l'application Express. Lors de ces validations, des jeux de données réels sont créés en amont puis supprimés en aval afin de ne pas polluer l'environnement de développement. Bien qu'un contrôleur puisse parfaitement fonctionner de manière isolée, il peut échouer lorsqu'il est connecté à la base de données réelle en raison,

par exemple, d'une contrainte de clé étrangère non respectée ou d'une erreur de format de date. Les tests d'intégration garantissent ainsi que des actions critiques, telles que la création d'un match depuis un créneau libre ou la participation à un match, fonctionnent de manière transactionnelle et fiable dans des conditions d'utilisation réelles.

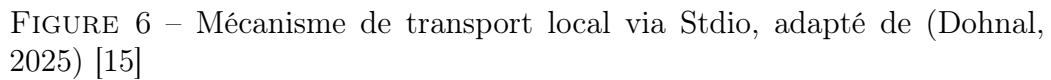
## 8.5 Maquettes

Maquettes de l'application, comment elle va être organisée, les différentes pages, etc..

## 8.6 Serveur MCP

### 8.6.1 Mécanisme de transport et communication

Le protocole MCP propose nativement deux mécanismes distincts pour acheminer les messages entre le client et le serveur. Le premier mécanisme, nommé Stdio, implique que le client MCP lance directement le serveur MCP en tant que sous-processus. Dans cette configuration, la communication s'effectue localement via les flux d'entrée standard (stdin) et de sortie standard (stdout) du système d'exploitation (Model Context protocol, 2025) [6], comme illustré par le diagramme de la figure 6.



6 mai 2026

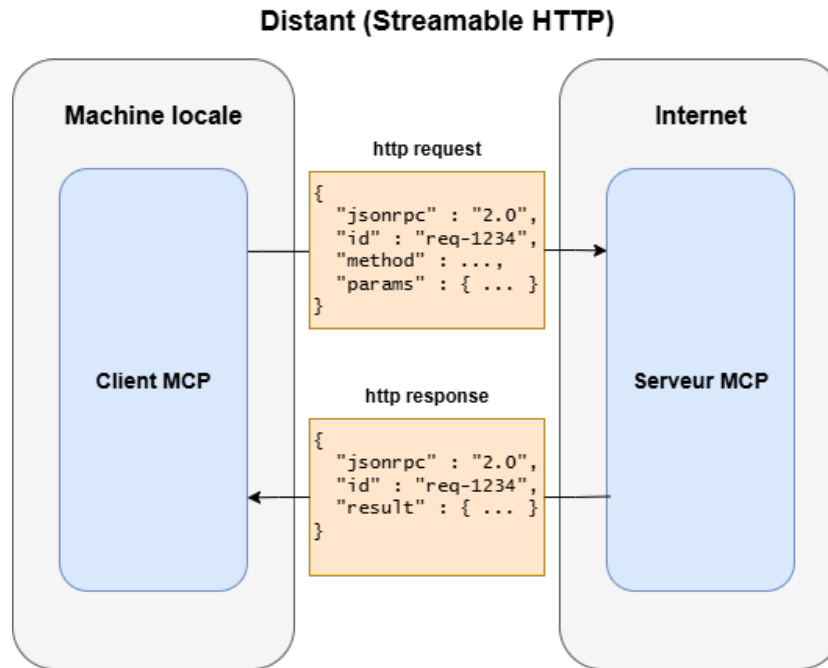


FIGURE 7 – Mécanisme de transport réseau via Streamable HTTP, adapté de (Dohnal, 2025) [15]

Pour le développement de l'écosystème PadelContext, le choix s'est délibérément porté sur le transport Streamable HTTP plutôt que sur le mécanisme Stdio. Cette décision architecturale repose sur la volonté de rendre le serveur MCP hautement disponible, évolutif et découplé de son client. Cette flexibilité garantit que le serveur pourra facilement être interrogé par d'autres interfaces, de nouveaux services ou de futurs agents d'intelligence artificielle, sans nécessiter de refonte de l'infrastructure logicielle.

Indépendamment du mécanisme de transport physique choisi, le format des échanges entre le client et le serveur est strictement régi par la spécification JSON-RPC 2.0. Il s'agit d'un protocole d'appel de procédure à distance léger, sans état, qui utilise le format de données JSON pour structurer les messages. Selon la spécification officielle, chaque requête envoyée par le client au serveur contient la version du protocole, un identifiant unique, le nom de la méthode à invoquer ainsi que ses paramètres d'exécution. Le serveur traite ensuite cette requête et retourne une réponse standardisée contenant soit le résultat

de l'opération, soit un objet d'erreur détaillé en cas d'échec (JSON-RPC, 2013) [4].

### 8.6.2 Définitions des outils MCP

Le serveur Model Context Protocol (MCP) constitue le pont de communication essentiel entre l'Assistant IA et les sources de données du système. Son rôle est d'exposer de manière standardisée les points de terminaison de l'API REST de PadelContext, ainsi que l'API externe de MeteoSwiss, sous forme d'outils (tools) directement exécutables par l'intelligence artificielle.

Afin de couvrir l'ensemble des cas d'usage définis lors de la modélisation fonctionnelle, le serveur MCP expose cinq outils distincts. Le tableau 10 résume leurs objectifs respectifs.

| Nom de l'outil                      | Source interrogée | Description fonctionnelle                                                                                                           |
|-------------------------------------|-------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <code>get-matches</code>            | API PadelContext  | Récupère la liste des matchs disponibles via le point de terminaison correspondant.                                                 |
| <code>get-available-slots</code>    | API PadelContext  | Récupère la liste des créneaux horaires disponibles via le point de terminaison correspondant.                                      |
| <code>create-match-from-slot</code> | API PadelContext  | Crée un match à partir d'un créneau horaire libre et y inscrit l'utilisateur authentifié via le point de terminaison correspondant. |
| <code>join-match</code>             | API PadelContext  | Inscrit l'utilisateur authentifié à un match existant via le point de terminaison correspondant.                                    |
| <code>get-meteo-swiss</code>        | API MeteoSwiss    | Fournit les prévisions climatiques locales pour valider la faisabilité d'une partie en extérieur.                                   |

TABLE 10 – Liste des outils exposés par le serveur MCP de PadelContext

Dans l'architecture du Model Context Protocol, la description sémantique d'une fonctionnalité est critique. Pour comprendre l'objectif et les spécificités d'un outil, les LLMs s'appuient exclusivement sur sa description en langage naturel. Une étude récente, ayant analysé 856 outils MCP, révèle que 97,1 % d'entre eux contiennent des défauts de conception qualifiés de « Tool Smells », avec 56 % des outils échouant même à définir clairement leur

objectif principal (Hasan et al. 2026, p. 4) [10]. Les auteurs soulignent que si ces descriptions sont défectueuses, sous-spécifiées ou trompeuses, l’agent IA risque de sélectionner le mauvais outil ou de fournir des arguments invalides, réduisant considérablement la fiabilité du système.

Bien que l’étude démontre que l’amélioration de ces descriptions permet d’augmenter le taux de succès des tâches, elle met également en garde sur le fait qu’une documentation trop volumineuse augmente le coût et le nombre d’étapes d’exécution. Il est donc nécessaire de trouver un juste équilibre (Hasan et al. 2026, p. 4) [10].

Pour PadelContext, les outils ont été rigoureusement structurés en s’appuyant sur la grille d’évaluation d’excellence (score de 5/5) proposée par l’étude mentionnée. Chaque description est structurée autour de six piliers sémantiques afin d’atteindre une qualité optimale (Hasan et al. 2026, p. 42) [10] :

1. **Purpose** : Explique clairement la fonction, le comportement et les données de retour de l’outil en utilisant un langage précis.
2. **Usage Guidelines** : Énonce explicitement les cas d’utilisation appropriés et les cas où l’outil ne doit pas être utilisé, incluant une désambiguïsation si le nom de l’outil s’avère ambigu.
3. **Limitation** : Indique clairement ce que l’outil ne retourne pas, les limites de sa portée ainsi que ses contraintes importantes.
4. **Parameter Explanation** : Chaque paramètre est détaillé avec son type, sa signification, son effet sur le comportement de l’outil, et son statut.
5. **Examples vs. Description Balance** : La description textuelle se suffit à elle-même, les exemples fournis servent uniquement à compléter l’explication et non à la remplacer.
6. **Length and Completeness** : La description de l’outil est composée de quatre phrases ou plus, rédigées dans un style concis et bien structuré, couvrant tous les aspects de son utilisation.

L’application stricte de cette grille garantit une réduction drastique des erreurs d’interprétation par l’agent IA, assurant ainsi une orchestration fiable des requêtes tout en optimisant la pertinence des résultats.

## 8.7 Cas d’usage

Exemple d’une réservation d’un terrain de padel en extérieur, avec un match-making automatique et une vérification de la météo en temps réel

## 8.8 Comparaison des résultats

Comparaison du temps et de facilité de réservation entre les plateformes traditionnelles et ma solution MCP.

# 9 Conclusion

## 9.1 Conclusion générale

## 9.2 Améliorations possibles

## Références

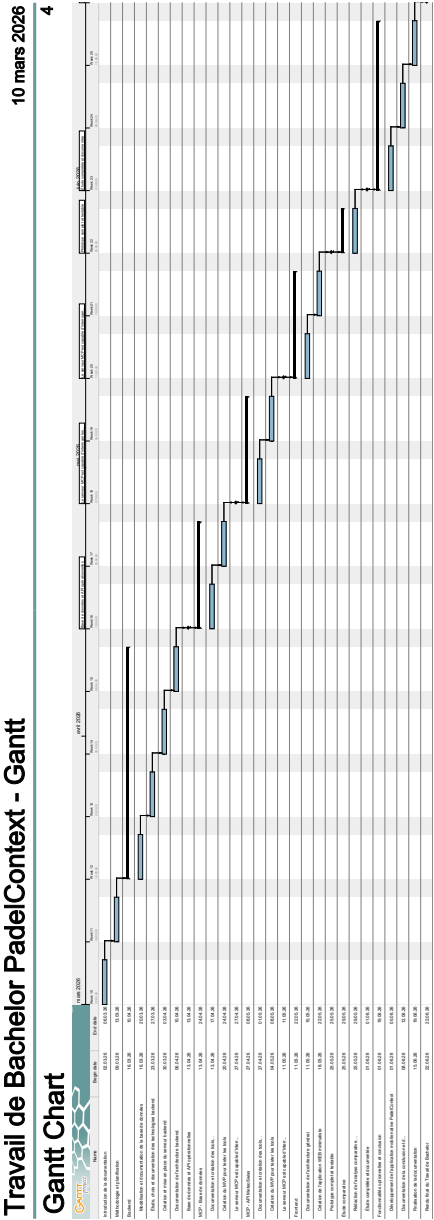
- [1] PlayPad | Consulte notre FAQ et trouve tes réponses. URL : <https://www.playpadapp.com/faqs>.
- [2] PostgreSQL : Versioning Policy. URL : <https://www.postgresql.org/support/versioning/>.
- [3] Technology | 2025 Stack Overflow Developer Survey. URL : <https://survey.stackoverflow.co/2025/technology/>.
- [4] JSON-RPC 2.0 Specification, January 2013. URL : <https://www.jsonrpc.org/specification>.
- [5] Node.js — Node.js Releases, October 2025. URL : <https://nodejs.org/en/about/previous-releases>.
- [6] Transports, November 2025. URL : <https://modelcontextprotocol.io/specification/2025-11-25/basic/transports>.
- [7] Sireesha Addanki. Architectural Shifts in API Design : The Progression from SOAP to REST and GraphQL. *IJSAT - International Journal on Science and Technology*, 16(2), June 2025. URL : <https://www.ijSAT.org/research-paper.php?id=6579>, doi: 10.71097/IJSAT.v16.i2.6579.

- [8] Agile Business. MoSCoW Prioritisation - DSDM Project Framework Handbook | Agile Business Consortium. URL : <https://www.agilebusiness.org/dsdm-project-framework/moscow-prioritisation.html>.
- [9] Anujkumarsinh Donvir. A Comparative Analysis of JavaScript Unit and End-to-End Testing Frameworks : Enhancing Quality Assurance in Modern Web Applications. In *2024 IEEE 16th International Conference on Computational Intelligence and Communication Networks (CICN)*, pages 1289–1296, December 2024. ISSN : 2472-7555. URL : <https://ieeexplore.ieee.org/document/10847365>, doi:10.1109/CICN63059.2024.10847365.
- [10] Mohammed Mehedi Hasan, Hao Li, Gopi Krishnan Rajbahadur, Bram Adams, and Ahmed E. Hassan. Model Context Protocol (MCP) Tool Descriptions Are Smelly! Towards Improving AI Agent Efficiency with Augmented MCP Tool Descriptions, February 2026. arXiv :2602.14878 [cs]. URL : <http://arxiv.org/abs/2602.14878>, doi:10.48550/arXiv.2602.14878.
- [11] Naga Murali Krishna Koneru. Containerization Best Practices- Using Docker and Kubernetes for Enterprise Applications. *Journal of Information Systems Engineering and Management*, 10(45s) :724–746, April 2025. URL : <https://jisem-journal.com/index.php/journal/article/view/8905>, doi:10.52783/jisem.v10i45s.8905.
- [12] Elena Lupu, Adriana Olteanu, and Anca Daniela Ionita. Concurrent Access Performance Comparison Between Relational Databases and Graph NoSQL Databases for Complex Algorithms. *Applied Sciences*, 14(21) :9867, January 2024. URL : <https://www.mdpi.com/2076-3417/14/21/9867>, doi:10.3390/app14219867.
- [13] Sanket Vilas Salunke and Abdelkader Ouda. A Performance Benchmark for the PostgreSQL and MySQL Databases. *Future Internet*, 16(10) :382, October 2024. URL : <https://www.mdpi.com/1999-5903/16/10/382>, doi:10.3390/fi16100382.
- [14] Suchetha Vijayakumar, Krishna Prasad K, and Raviraja Holla M. Assessing the Effectiveness of MoSCoW Prioritization in Software Development : A Holistic Analysis across Methodologies. *EAI Endorsed Transactions on Internet of Things*, 10, October 2024. URL : <https://publications.eai.eu/index.php/IoT/article/view/6515>, doi:10.4108/eetiot.6515.

- [15] Tom Dohnal. Build A Remote MCP Server With TypeScript (Streamable HTTP), July 2025. URL : <https://www.youtube.com/watch?v=nTPeM813zTg>.
- [16] Serhii Zhadko-Bazilevych. Analysis of ORM framework approaches for Node.js. *Journal of Computer Sciences Institute*, 37 :426–430, December 2025. URL : <https://ph.pollub.pl/index.php/jcsi/article/view/7951>, doi:10.35784/jcsi.7951.

10 Annexes

10.1 Diagramme gantt du projet PadelContext



## 10.2 Documentation Swagger de l'API PadelContext

28/04/2026 22:26

Swagger UI

# Padel Context API

1.0.0OAS 3.0

API du travail de bachelor Padel Context

Authorize

### Authentication

POST

/api/auth/login

Authentifier un utilisateur

Permet à un utilisateur de s'authentifier en fournissant son email et son mot de passe. Retourne un token JWT et les informations de l'utilisateur.

Parameters

Try it out

No parameters

Request body required

application/json

Les identifiants de l'utilisateur

Example ValueSchema

```
{  "email": "jonathan.borel@padelcontext.com",  "password": "pomme123"}
```

Responses

localhost:3000/api-docs/#/Matches/post\_api\_matches\_\_matchId\_\_join

1/13

28/04/2026 22:26

Swagger UI

| Code | Description                                                                                                                                                                                                                                                                                                                                                                                                               | Links    |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| 200  | Authentification réussie                                                                                                                                                                                                                                                                                                                                                                                                  | No links |
|      | <div>Media type</div> <div>application/json</div> <div>Controls Accept header.</div> <div>Example ValueSchema</div> <div><pre>{  "message": "login successful",  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",  "user": {    "id": "123e4567-e89b-12d3-a456-426614174000",    "firstname": "John",    "lastname": "Doe",    "email": "jonathan.borel@padelcontext.com",    "level": "intermédiaire"  }}</pre></div> |          |
| 400  | Erreur de validation - Email ou mot de passe manquants ou invalides                                                                                                                                                                                                                                                                                                                                                       | No links |
|      | <div>Media type</div> <div>application/json</div> <div>Examples</div> <div>Paramètres manquants</div> <div>Example ValueSchema</div> <div><pre>{  "message": "email and password are required"}</pre></div>                                                                                                                                                                                                               |          |
| 401  | Authentification échouée - Email non trouvé ou mot de passe incorrect                                                                                                                                                                                                                                                                                                                                                     | No links |
|      | <div>Media type</div> <div>application/json</div> <div>Examples</div> <div>Identifiants invalides</div> <div>Example ValueSchema</div> <div><pre>{  "message": "invalid credentials"}</pre></div>                                                                                                                                                                                                                         |          |
| 500  | Erreur serveur interne                                                                                                                                                                                                                                                                                                                                                                                                    | No links |
|      | <div>Media type</div> <div>application/json</div> <div>Example ValueSchema</div> <div><pre>{  "message": "Error while logging in"}</pre></div>                                                                                                                                                                                                                                                                            |          |

localhost:3000/api-docs/#!/Matches/post\_api\_matches\_\_matchId\_\_join

2/13

28/04/2026 22:26

Swagger UI

| Code | Description | Links |
|------|-------------|-------|
|------|-------------|-------|

### Available Slots

GET

/api/available-slots

Récupérer les créneaux disponibles à l'aide de filtres

Retourne les créneaux disponibles pour chaque terrain correspondant aux filtres.

Sans l'utilisation de `timeFrom` et `timeTo`, retourne les créneaux pour les 7 prochains jours.

Les créneaux déjà occupés par des matchs ouverts et complétés sont exclus. Les matchs annulés ne bloquent pas un créneau.

Les filtres query sont optionnels et combinables.

Parameters

Try it out

| Name                                   | Description                                                                                                                                                                                  |
|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| city<br>string<br>(query)              | Ville du club.<br><br>Example : Lancy<br><div>Lancy</div>                                                                                                                                    |
| courtType<br>string<br>(query)         | Type de terrain. La valeur est normalisée en majuscules. Une valeur hors enum est ignorée.<br><br>Available values : INDOOR, OUTDOOR, COVERED<br><br>Example : COVERED<br><div>COVERED</div> |
| hasEquipmentBox<br>boolean<br>(query)  | Filtrer selon la présence d'une box d'équipement.<br><br>Example : true<br><div>true</div>                                                                                                   |
| minPricePerPerson<br>number<br>(query) | Prix minimum par personne (inclus).<br><br>Example : 10<br><div>10</div>                                                                                                                     |
| maxPricePerPerson<br>number            | Prix maximum par personne (inclus).                                                                                                                                                          |

localhost:3000/api-docs/#!/Matches/post\_api\_matches\_\_matchId\_\_join

3/13

28/04/2026 22:26

Swagger UI

| Name                                              | Description                                                                                    |
|---------------------------------------------------|------------------------------------------------------------------------------------------------|
| <i>(query)</i>                                    | <i>Example : 25</i>                                                                            |
|                                                   | <input type="text" value="25"/>                                                                |
| slotDuration<br>integer<br><i>(query)</i>         | Durée exacte du créneau en minutes.<br><i>Example : 60</i>                                     |
|                                                   | <input type="text" value="60"/>                                                                |
| minSlotDuration<br>integer<br><i>(query)</i>      | Durée minimale du créneau en minutes (incluse).<br><i>Example : 45</i>                         |
|                                                   | <input type="text" value="45"/>                                                                |
| maxSlotDuration<br>integer<br><i>(query)</i>      | Durée maximale du créneau en minutes (incluse).<br><i>Example : 120</i>                        |
|                                                   | <input type="text" value="120"/>                                                               |
| timeFrom<br>string(\$date-time)<br><i>(query)</i> | Début de la fenêtre de disponibilité à retourner.<br><i>Example : 2026-04-15T08:00:00.000Z</i> |
|                                                   | <input type="text" value="2026-04-15T08:00:00.000Z"/>                                          |
| timeTo<br>string(\$date-time)<br><i>(query)</i>   | Fin de la fenêtre de disponibilité à retourner.<br><i>Example : 2026-04-15T20:00:00.000Z</i>   |
|                                                   | <input type="text" value="2026-04-15T20:00:00.000Z"/>                                          |

Responses

| Code | Description                                         | Links    |
|------|-----------------------------------------------------|----------|
| 200  | Liste des terrains avec leurs créneaux disponibles. | No links |

Media type

Controls Accept header.

Example Value

Schema

Examples

localhost:3000/api-docs/#!/Matches/post\_api\_matches\_\_matchId\_\_join

4/13

28/04/2026 22:26

Swagger UI

| Code | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Links    |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
|      | <pre>[   {     "court": {       "id": 11,       "name": "Court Alpha",       "type": "INDOOR",       "hasEquipmentBox": true,       "pricePerPerson": 18,       "slotDuration": 60,       "club": {         "name": "Geneva Club",         "city": "Lancy",         "openingTime": "08:00",         "closingTime": "10:00"       }     },     "availableSlots": [       {         "startTime": "2026-04-15T09:00:00.000Z",         "endTime": "2026-04-15T10:00:00.000Z"       }     ]   } ]</pre> |          |
| 400  | Fenêtre invalide ( <code>timeTo</code> <= <code>timeFrom</code> ).<br><br>Media type<br><div>application/json</div><br>Example Value   Schema<br><pre>{   "message": "timeTo must be greater than timeFrom" }</pre>                                                                                                                                                                                                                                                                                | No links |
| 500  | Erreur interne lors de la récupération des créneaux disponibles.<br><br>Media type<br><div>application/json</div><br>Example Value   Schema<br><pre>{   "message": "Error while fetching available slots" }</pre>                                                                                                                                                                                                                                                                                  | No links |

### Matches

GET

/api/matches

Récupérer les matchs ouverts à l'aide de filtres

Retourne uniquement les matchs ouverts à venir, soit un match avec au moins une place disponible.

localhost:3000/api-docs/#!/Matches/post\_api\_matches\_\_matchId\_\_join

5/13

28/04/2026 22:26

Swagger UI

Les filtres query sont optionnels et combinables.

Parameters

Try it out

| Name                                   | Description                                                                                                                                                                                      |
|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| city<br>string<br>(query)              | Ville du club.<br><i>Example</i> : Lancy<br><div>Lancy</div>                                                                                                                                     |
| courtType<br>string<br>(query)         | Type de terrain. La valeur est normalisée en majuscules. Une valeur hors enum est ignorée.<br><i>Available values</i> : INDOOR, OUTDOOR, COVERED<br><i>Example</i> : INDOOR<br><div>INDOOR</div> |
| hasEquipmentBox<br>boolean<br>(query)  | Filtrer selon la présence d'une box d'équipement.<br><i>Example</i> : true<br><div>true</div>                                                                                                    |
| minPricePerPerson<br>number<br>(query) | Prix minimum par personne (inclus).<br><i>Example</i> : 10<br><div>10</div>                                                                                                                      |
| maxPricePerPerson<br>number<br>(query) | Prix maximum par personne (inclus).<br><i>Example</i> : 20<br><div>20</div>                                                                                                                      |
| slotDuration<br>integer<br>(query)     | Durée exacte du créneau en minutes.<br><i>Example</i> : 120<br><div>120</div>                                                                                                                    |
| minSlotDuration<br>integer<br>(query)  | Durée minimale du créneau en minutes (incluse).<br><i>Example</i> : 90<br><div>90</div>                                                                                                          |

localhost:3000/api-docs/#!/Matches/post\_api\_matches\_\_matchId\_\_join

6/13

28/04/2026 22:26

Swagger UI

| Name                                            | Description                                                                                                                                                                          |
|-------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| maxSlotDuration<br>integer<br>(query)           | Durée maximale du créneau en minutes (incluse).<br>Example : 120<br><div>120</div>                                                                                                   |
| availableSpots<br>integer<br>(query)            | Nombre exact de places disponibles.<br>Example : 2<br><div>2</div>                                                                                                                   |
| minAvailableSpots<br>integer<br>(query)         | Nombre minimal de places disponibles (inclus).<br>Example : 1<br><div>1</div>                                                                                                        |
| startTimeFrom<br>string(\$date-time)<br>(query) | Date/heure minimale de début. Si la valeur est dans le passé, la borne utilisée reste la date actuelle.<br>Example : 2026-04-15T18:00:00.000Z<br><div>2026-04-15T18:00:00.000Z</div> |
| startTimeTo<br>string(\$date-time)<br>(query)   | Date/heure maximale de début (incluse).<br>Example : 2026-04-16T20:00:00.000Z<br><div>2026-04-16T20:00:00.000Z</div>                                                                 |
| endTimeFrom<br>string(\$date-time)<br>(query)   | Date/heure minimale de fin (incluse).<br>Example : 2026-04-15T19:00:00.000Z<br><div>2026-04-15T19:00:00.000Z</div>                                                                   |
| endTimeTo<br>string(\$date-time)<br>(query)     | Date/heure maximale de fin (incluse).<br>Example : 2026-04-15T21:00:00.000Z<br><div>2026-04-15T21:00:00.000Z</div>                                                                   |
| participantAverageLevel<br>number<br>(query)    | Niveau moyen cible des participants déjà inscrits.<br>Example : 5                                                                                                                    |

localhost:3000/api-docs/#!/Matches/post\_api\_matches\_\_matchId\_\_join

7/13

28/04/2026 22:26

Swagger UI

| Name                             | Description                                                                                                                                                                                      |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| participantAverageLevelTolerance | <div>5</div> <div>Tolérance absolue autour de <b>participantAverageLevel</b> .</div> <div>number</div> <div>(query)</div> <div>Default value : 0.5</div> <div>Example : 0.1</div> <div>0.1</div> |

Responses

| Code | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | Links    |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| 200  | <div>Liste des matchs correspondant aux filtres.</div> <div>Media type</div> <div>application/json</div> <div>Controls Accept header.</div> <div>Example Value</div> <div>Schema</div> <div>[<br/>  {<br/>    "id": 101,<br/>    "startTime": "2026-04-15T18:00:00.000Z",<br/>    "endTime": "2026-04-15T20:00:00.000Z",<br/>    "status": "OPEN",<br/>    "availableSpots": 2,<br/>    "count": {<br/>      "name": "Court Alpha",<br/>      "type": "INDOOR",<br/>      "hasEquipmentBox": true,<br/>      "pricePerPerson": 18,<br/>      "slotDuration": 120,<br/>      "club": {<br/>        "name": "Geneva Club",<br/>        "city": "Lancy",<br/>        "openingTime": "08:00",<br/>        "closingTime": "23:00"<br/>      }<br/>    }<br/>  },<br/>  "participants": [<br/>    {<br/>      "user": {<br/>        "firstname": "Jonathan",<br/>        "lastname": "Borel-Jaquet",<br/>        "email": "jonathan.borel@padelcontext.com",<br/>        "level": 2<br/>      }<br/>    }<br/>  ]<br/>]</div> | No links |
| 500  | <div>Erreur interne lors de la récupération des matchs.</div> <div>Media type</div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | No links |

localhost:3000/api-docs/#!/Matches/post\_api\_matches\_\_matchId\_\_join

8/13

28/04/2026 22:26

Swagger UI

| Code | Description                                                                                                                              | Links |
|------|------------------------------------------------------------------------------------------------------------------------------------------|-------|
|      | <div>application/json</div> <div>Example ValueSchema</div> <div><pre>{<br/>  "message": "Error while fetching matches"<br/>}</pre></div> |       |

POST

/api/matches/from-slot

Créer un match depuis un créneau disponible

Permet à un utilisateur authentifié de créer un match à partir d'un slot disponible, puis de s'y inscrire automatiquement.

Parameters

Try it out

No parameters

Request body required

application/json

Informations du créneau à transformer en match.  
Example ValueSchema

```
{  
  "courtId": 12,  
  "startTime": "2026-04-15T18:00:00.000Z",  
  "endTime": "2026-04-15T19:00:00.000Z"  
}
```

Responses

| Code | Description                                                                                                                                  | Links    |
|------|----------------------------------------------------------------------------------------------------------------------------------------------|----------|
| 201  | Match créé avec succès depuis le créneau.<br><div>Media type<div>application/json</div>Controls Accept header.<br/>Example ValueSchema</div> | No links |

localhost:3000/api-docs/#!/Matches/post\_api\_matches\_\_matchId\_\_join

9/13

28/04/2026 22:26

Swagger UI

Code

```
"availableSpots": 3,
"startTime": "2026-04-15T18:00:00.000Z",
"endTime": "2026-04-15T19:00:00.000Z",
"creator_id": 12,
"court": {
  "name": "Court Alpha",
  "type": "INDOOR",
  "club": {
    "name": "Geneva Club",
    "city": "Lancy"
  }
},
"participants": [
  {
    "user": {
      "id": 21,
      "firstname": "Jonathan",
      "lastname": "Borel-Jaquet",
      "email": "jonathan.borel@padelcontext.com",
      "level": 2
    },
    "joinedAt": "2026-04-15T17:30:00.000Z"
  }
]
},
]
```

Links

400

Données invalides ou créneau non conforme aux règles métier.

No links

Media type

Examples

application/json

Paramètres requis manquants

Example Value

Schema

{
 "message": "courtId, startTime and endTime are required"
}

401

Utilisateur non authentifié.

No links

Media type

application/json

Example Value

Schema

{
 "message": "unauthorized"
}

404

Terrain introuvable.

No links

Media type

application/json

Example Value

Schema

{
 "message": "court not found"
}

localhost:3000/api-docs/#!/Matches/post\_api\_matches\_\_matchId\_\_join

10/13

28/04/2026 22:26

Swagger UI

| Code | Description                                                                                                                                                                                                    | Links    |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| 409  | Le créneau n'est plus disponible.<br><br>Media type<br>application/json<br><br>Example Value Schema<br><pre>{<br/>  "message": "slot is no longer available"<br/>}</pre>                                       | No links |
| 500  | Erreur interne lors de la création du match depuis le créneau.<br><br>Media type<br>application/json<br><br>Example Value Schema<br><pre>{<br/>  "message": "Error while creating match from slot"<br/>}</pre> | No links |

POST

/api/matches/{matchId}/join

Rejoindre un match ouvert



Permet à un utilisateur authentifié de rejoindre un match ouvert s'il reste des places disponibles.

Parameters

Try it out

| Name                                                          | Description                                                              |
|---------------------------------------------------------------|--------------------------------------------------------------------------|
| <b>matchId</b> <small>* required</small><br>integer<br>(path) | Identifiant du match à rejoindre.<br><br>Example : 101<br><div>101</div> |

Responses

localhost:3000/api-docs/#!/Matches/post\_api\_matches\_\_matchId\_\_join

11/13

28/04/2026 22:26

Swagger UI

| Code | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | Links    |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| 200  | Match rejoint avec succès.<br><br>Media type<br>application/json<br><br>Controls Accept header.<br><br>Example Value   Schema<br><pre>{   "message": "successfully joined match",   "match": {     "id": 101,     "status": "OPEN",     "availableSpots": 1,     "startTime": "2026-04-15T18:00:00.000Z",     "endTime": "2026-04-15T20:00:00.000Z",     "creator_id": 12,     "court": {       "name": "Court Alpha",       "type": "INDOOR",       "club": {         "name": "Geneva Club",         "city": "Lancy"       }     }   },   "participants": [     {       "user": {         "id": 21,         "firstname": "Jonathan",         "lastname": "Borel-Jaquet",         "email": "jonathan.borel@padelcontext.com",         "level": 2       },       "joinedAt": "2026-04-15T17:30:00.000Z"     }   ] }</pre> | No links |
| 400  | Identifiant de match invalide, match fermé, plus de place indisponible ou utilisateur déjà inscrit.<br><br>Media type   Examples<br>application/json   Identifiant invalide<br><br>Example Value   Schema<br><pre>{   "message": "invalid match ID" }</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | No links |
| 401  | Utilisateur non authentifié.<br><br>Media type<br>application/json<br><br>Example Value   Schema<br><pre>{   "message": "unauthorized" }</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | No links |

localhost:3000/api-docs/#!/Matches/post\_api\_matches\_\_matchId\_\_join

12/13

28/04/2026 22:26Swagger UI

| Code | Description                                                                                                                                                                                                 | Links    |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| 404  | Match introuvable. <div>Media type<div>application/json</div></div> <div>Example ValueSchema<div><div>{<br/>  "message": "match not found"<br/>}</div></div></div>                                          | No links |
| 500  | Erreur interne lors de la participation au match. <div>Media type<div>application/json</div></div> <div>Example ValueSchema<div><div>{<br/>  "message": "Error while joining match"<br/>}</div></div></div> | No links |